THE PARTIAL SUMS OF $\mathrm{fusc}(k)/\mathrm{fusc}(k+1)$ ARE INTEGERS AT $n = 2 \cdot 4^k - 1$ AND $n = 2 \cdot 4^k - 2$: A PROOF OF THE CONJECTURE IN OEIS A355076

MATH0

Abstract. Let fusc be Stern's diatomic sequence and $s(n) = \sum_{k=0}^{n} \mathrm{fusc}(k)/\mathrm{fusc}(k+1)$. We prove the conjecture in OEIS A355076 (R. Sigrist, 2022) that the denominator of $s(n)$ is 1, that is, $s(n)$ is an integer, for every n of the form $2 \cdot 4^k - 1$ or $2 \cdot 4^k - 2$ with $k \geq 0$; in fact $s(2 \cdot 4^k - 1) = 3 \cdot 4^k - k - 2$ and $s(2 \cdot 4^k - 2) = 3 \cdot 4^k - 3k - 3$. The proof rests on the closed form $\sum_{k<2^M} \mathrm{fusc}(k)/\mathrm{fusc}(k+1) = (3 \cdot 2^M - 3 - M)/2$, which follows by summation from the known value $(3 \cdot 2^m - 1)/2$ of the sum of the m-th row of the Calkin–Wilf tree (proved by K. S. Choudary and A. S. Reddy in 2020; its numerator $3 \cdot 2^m - 1$ was recorded by C. Najafi in OEIS A052940 in 2011). The conjecture is thus an immediate consequence of published results, and we classify this note as the first formal proof of a known result. The proof is verified in Lean 4 against a statement that math0 formalized from the OEIS entry, whose definitions are checked by the kernel against the listed terms of A355076 and A002487, and which was reviewed for fidelity and frozen before the proof attempt.

1. Introduction

Stern's diatomic sequence [8], called fusc by Dijkstra [3], is OEIS A002487 [9]: $\mathrm{fusc}(0) = 0$, $\mathrm{fusc}(1) = 1$, $\mathrm{fusc}(2n) = \mathrm{fusc}(n)$ and $\mathrm{fusc}(2n+1) = \mathrm{fusc}(n) + \mathrm{fusc}(n+1)$, with first terms $0, 1, 1, 2, 1, 3, 2, 3, 1, 4, \ldots$. The ratios $x_j = \mathrm{fusc}(j)/\mathrm{fusc}(j+1)$, $j \geq 1$, list every positive rational number exactly once, in reduced form [8, 1]. Calkin and Wilf [1] showed that they are the breadth-first enumeration of the Calkin–Wilf tree, the complete binary tree with root $1/1$ in which a vertex a/b has left child $a/(a+b)$ and right child $(a+b)/b$; level m of the tree (root at level 0) consists of the x_j with $2^m \leq j < 2^{m+1}$. We also set $x_0 = 0$. For background see the survey [7].

The OEIS entry A355076 [12], created by Rémy Sigrist on June 18, 2022, lists the denominators of the partial sums $s(n) = \sum_{k=0}^{n} x_k$; A355075 [11] lists the numerators. For instance $s(4) = 0 + 1 + \frac{1}{2} + 2 + \frac{1}{3} = \frac{23}{6}$. The entry conjectures that $s(n)$ is an integer for $n = 2 \cdot 4^k - 1$ and $n = 2 \cdot 4^k - 2$; the listed terms $a(0), \ldots, a(52)$ equal 1 exactly at $n = 0, 1, 6, 7, 30, 31$. The current revision of the entry (#11, June 19, 2022; checked in October 2026) still states the conjecture without a proof or a reference.

Prior work. The conjecture follows at once from three known facts. (1) *Row sums*: the sum of level m of the Calkin–Wilf tree is $(3 \cdot 2^m - 1)/2$. Its numerator was recorded by Carl Najafi in a comment of July 10, 2011 on OEIS A052940 [10], a sequence with $a(0) = 1$ and $a(n) = 3 \cdot 2^n - 1$ for $n \geq 1$, which the comment describes as the "numerator of the sum of terms at the n-th level of the Calkin–Wilf tree"; the comment states neither the denominator nor a proof. The formula was proved by Choudary and Reddy [2, Theorem 1.1, Part 11] in the form $3 \cdot 2^{n-2} - \frac{1}{2}$, with the root at level 1; their inductive proof pairs each vertex a/b of a level with its reciprocal b/a, which lies on the same level [2, Theorem 1.1, Part 4], and our proof uses the same pairing. (2) *Row symmetry*: each row $\mathrm{fusc}(2^m), \ldots, \mathrm{fusc}(2^{m+1})$ is a palindrome, as recorded in A002487 in a comment by Alexander Adamchuk (October 10, 2006) [9]; this is equivalent to the reciprocal

Date: October 18, 2026.

symmetry of the levels. (3) *Extreme values*: $\operatorname{fusc}(2^M) = 1$ and $\operatorname{fusc}(2^M - 1) = M$, also in Adamchuk's comment.

Summing (1) over the levels $0, \dots, M-1$ gives $s(2^M - 1) = (3 \cdot 2^M - 3 - M)/2$, an integer for odd M, and subtracting the last term $x_{2^M-1} = M$ given by (3) handles $n = 2^M - 2$. We found no place where this deduction is recorded or connected with A355076, and the referees of this note found no resolution of the conjecture in the OEIS or elsewhere. Thus the conjecture had not been settled in print, but all of its mathematical content is in the results above, and we classify the result as known.

The conjecture is not among the OEIS statements of the formal-conjectures repository [4] (checked at commit `c77fdd4`), and the referees found no formalization of the Calkin–Wilf row sums in Lean or Mathlib. What this note adds is a self-contained proof that follows a kernel-checked Lean proof of a frozen formal statement of the conjecture (Section 4); to our knowledge this is the first formal proof of the statement.

2. Statement

Definition 1. Let $\operatorname{fusc}\colon \mathbb{N} \to \mathbb{N}$ be determined by $\operatorname{fusc}(0) = 0$, $\operatorname{fusc}(1) = 1$ and, for all $n \in \mathbb{N}$, $\operatorname{fusc}(2n) = \operatorname{fusc}(n)$ and $\operatorname{fusc}(2n+1) = \operatorname{fusc}(n) + \operatorname{fusc}(n+1)$. For $j \in \mathbb{N}$ let $x_j = \operatorname{fusc}(j)/\operatorname{fusc}(j+1) \in \mathbb{Q}$, let $s(n) = \sum_{k=0}^{n} x_k$, and let $a(n) = \operatorname{den} s(n) \geq 1$ be the denominator of $s(n)$ in lowest terms.

These equations determine fusc: at every argument ≥ 2 they express fusc through values at smaller arguments, and for $n = 0$ they read $\operatorname{fusc}(0) = \operatorname{fusc}(0)$ and $\operatorname{fusc}(1) = \operatorname{fusc}(0) + \operatorname{fusc}(1)$, which are consistent with the initial values. By Lemma 3 below, $\operatorname{fusc}(j+1) > 0$, so x_j is well defined. The entry's name and its conjecture, stated in the FORMULA section by the entry's author Rémy Sigrist (June 18, 2022) [12], read verbatim:

> NAME: `a(n) is the denominator of Sum_{k = 0..n} fusc(k)/fusc(k+1) (where fusc is Stern's diatomic series A002487).`
>
> FORMULA: `Conjecture: a(n) = 1 for n of the form 2*4^k - 1 or 2*4^k - 2 for some k >= 0.`

Theorem 2. *For every $n \in \mathbb{N}$ such that $n = 2 \cdot 4^k - 1$ or $n = 2 \cdot 4^k - 2$ for some $k \in \mathbb{N}$, we have $a(n) = 1$; that is, $s(n)$ is an integer.*

The proof gives the values explicitly: $s(2 \cdot 4^k - 1) = 3 \cdot 4^k - k - 2$ and $s(2 \cdot 4^k - 2) = 3 \cdot 4^k - 3k - 3$ for every $k \geq 0$ (Proposition 10). For example, $s(6) = 0 + 1 + \frac{1}{2} + 2 + \frac{1}{3} + \frac{3}{2} + \frac{2}{3} = 6$ and $s(7) = s(6) + x_7 = 6 + 3 = 9$, the case $k = 1$.

Lean formalization. The statement was formalized by math0 in `lean/Math0/Statements/OEIS/A355076.lean`, reviewed for fidelity, and frozen before any proof attempt (SHA-256 hash beginning `e42856b0`). Its definitions and the statement are reproduced verbatim below, except that the copyright comment, the line `import Mathlib`, docstrings, blank lines, the lemmas `fuscAux_eq_fuscAux`, `fuscAux_succ` and `fuscAux_eq_fusc`, and the proofs of `fusc_even` and `fusc_odd` are omitted.

```
namespace Math0.Statements.A355076
def fuscAux : ℕ → ℕ → ℕ
  | 0, _ => 0
  | fuel + 1, n =>
    if n ≤ 1 then n
    else if n % 2 = 0 then fuscAux fuel (n / 2)
    else fuscAux fuel (n / 2) + fuscAux fuel (n / 2 + 1)
def fusc (n : ℕ) : ℕ := fuscAux (n + 1) n
```

```
theorem fusc_zero : fusc 0 = 0 := by decide +kernel
theorem fusc_one : fusc 1 = 1 := by decide +kernel
theorem fusc_even (n : ℕ) : fusc (2 * n) = fusc n
theorem fusc_odd (n : ℕ) : fusc (2 * n + 1) = fusc n + fusc (n + 1)
theorem fusc_data : (List.range 92).map fusc =
    [0, 1, 1, 2, 1, 3, 2, 3, 1, 4, 3, 5, 2, 5, 3, 4, 1, 5, 4, 7, 3, 8, 5, 7, 2, 7, 5, 8, 3, 7,
     4, 5, 1, 6, 5, 9, 4, 11, 7, 10, 3, 11, 8, 13, 5, 12, 7, 9, 2, 9, 7, 12, 5, 13, 8, 11, 3,
     10, 7, 11, 4, 9, 5, 6, 1, 7, 6, 11, 5, 14, 9, 13, 4, 15, 11, 18, 7, 17, 10, 13, 3, 14, 11,
     19, 8, 21, 13, 18, 5, 17, 12, 19] := by
  decide +kernel
def a (n : ℕ) : ℕ :=
  (∑ k ∈ Finset.range (n + 1), ((fusc k : ℚ) / (fusc (k + 1) : ℚ))).den
theorem a_data : (List.range 53).map a =
    [1, 1, 2, 2, 6, 3, 1, 1, 4, 12, 60, 60, 12, 4, 2, 2, 10, 20, 140, 420, 840, 840, 840, 840,
     840, 840, 420, 140, 20, 5, 1, 1, 6, 30, 90, 180, 1980, 13860, 13860, 13860, 13860, 27720,
     360360, 72072, 72072, 72072, 8008, 8008, 72072, 72072, 72072, 360360, 27720] := by
  decide +kernel
def statement : Prop :=
  ∀ n : ℕ, (∃ k : ℕ, n = 2 * 4 ^ k - 1 ∨ n = 2 * 4 ^ k - 2) → a n = 1
end Math0.Statements.A355076
```

Mathlib has no fusc, so `fusc` is computed by the defining recursion with a fuel parameter, which makes it structurally recursive and lets the kernel evaluate it. With fuel $n+1$ the fuel never runs out, since the recursive calls are on $\lfloor n/2 \rfloor < n$ and $\lfloor n/2 \rfloor + 1 < n$ for $n \geq 2$. The omitted lemma `fuscAux_eq_fuscAux` shows that the value does not depend on the fuel once it exceeds the argument; from it the file proves `fusc_even` and `fusc_odd` for every n, while `fusc_zero` and `fusc_one` are checked by `decide +kernel`. These four theorems are the defining equations of A002487, which determine the sequence. So `fusc` is fusc.

`a n` is Mathlib's `Rat.den`, the positive denominator in lowest terms, of the exact rational sum $\sum_{k=0}^{n} \mathrm{fusc}(k)/\mathrm{fusc}(k+1)$; as $\mathrm{fusc}(k+1) > 0$, Lean's convention $x/0 = 0$ plays no role. The statement is the conjecture read literally. Since $2 \cdot 4^k \geq 2$, the truncated subtractions in $\mathbb{N}$ are exact; $k = 0$ gives $n = 1$ and $n = 0$. Like the OEIS formula, it asserts only one direction (see Remark 11).

The theorems `fusc_data` and `a_data`, proved by `decide +kernel`, check that the definitions reproduce the 92 terms of A002487 and all 53 terms of A355076 listed in the OEIS; the fidelity reviewer also compared both lists with the OEIS data.

3. Proof

We follow the Lean proof; each lemma is labelled with the name of the corresponding private Lean lemma. The proof uses only the four defining equations of fusc (in Lean, `fusc_zero`, `fusc_one`, `fusc_even`, `fusc_odd`, together with the base cases $\mathrm{fusc}(0) = 0$, $\mathrm{fusc}(1) = 1$ and $\mathrm{fusc}(2) = 1$, which the kernel also evaluates directly; see Section 4).

Lemma 3 (`fusc_pos`)**.** $\mathrm{fusc}(n) > 0$ *for every* $n \geq 1$.

Proof. Strong induction on n. If $n = 2k$, then $1 \leq k < n$ and $\mathrm{fusc}(n) = \mathrm{fusc}(k) > 0$. If $n = 1$, then $\mathrm{fusc}(n) = 1$. If $n = 2k+1$ with $k \geq 1$, then $\mathrm{fusc}(n) = \mathrm{fusc}(k) + \mathrm{fusc}(k+1) \geq \mathrm{fusc}(k) > 0$, since $1 \leq k < n$. □

Lemma 4 (`fusc_symm`)**.** *For all* $m, i, j \in \mathbb{N}$ *with* $i + j = 2^m$ *we have* $\mathrm{fusc}(2^m + i) = \mathrm{fusc}(2^m + j)$.

Proof. Induction on m. For $m = 0$, $\{i, j\} = \{0, 1\}$ and the claim is $\mathrm{fusc}(1) = \mathrm{fusc}(2)$; both equal 1. Assume the claim for m and let $i + j = 2^{m+1} = 2 \cdot 2^m$. Then i and j have the same parity. If $i = 2i'$ and $j = 2j'$, then $i' + j' = 2^m$, and $\mathrm{fusc}(2^{m+1} + 2i') = \mathrm{fusc}(2(2^m + i')) = \mathrm{fusc}(2^m + i') =$

$\mathrm{fusc}(2^m+j') = \mathrm{fusc}(2^{m+1}+2j')$. If $i = 2i'+1$ and $j = 2j'+1$, then $i'+(j'+1) = (i'+1)+j' = 2^m$, so the induction hypothesis gives $\mathrm{fusc}(2^m+i') = \mathrm{fusc}(2^m+j'+1)$ and $\mathrm{fusc}(2^m+i'+1) = \mathrm{fusc}(2^m+j')$. Hence

$$\begin{aligned}\mathrm{fusc}(2^{m+1}+2i'+1) &= \mathrm{fusc}(2^m+i') + \mathrm{fusc}(2^m+i'+1)\\ &= \mathrm{fusc}(2^m+j'+1) + \mathrm{fusc}(2^m+j') = \mathrm{fusc}(2^{m+1}+2j'+1),\end{aligned}$$

using $2^{m+1}+2i'+1 = 2(2^m+i')+1$ and the recursion twice. □

Equivalently, $\mathrm{fusc}(2^m+i) = \mathrm{fusc}(2^{m+1}-i)$ for $0 \le i \le 2^m$: the m-th row is a palindrome.

Lemma 5 (`fusc_two_pow`, `fusc_two_pow_sub_one`)**.** *For every $M \in \mathbb{N}$, $\mathrm{fusc}(2^M) = 1$ and $\mathrm{fusc}(2^M-1) = M$.*

Proof. Induction on M: $\mathrm{fusc}(2^0) = \mathrm{fusc}(1) = 1$ and $\mathrm{fusc}(2^{M+1}) = \mathrm{fusc}(2\cdot 2^M) = \mathrm{fusc}(2^M)$; further $\mathrm{fusc}(2^0-1) = \mathrm{fusc}(0) = 0$, and since $2^{M+1}-1 = 2(2^M-1)+1$, we get $\mathrm{fusc}(2^{M+1}-1) = \mathrm{fusc}(2^M-1)+\mathrm{fusc}(2^M) = M+1$. □

Lemma 6 (`ratio_even`, `ratio_odd`)**.** *For every $j \in \mathbb{N}$, $x_{2j} = \mathrm{fusc}(j)/(\mathrm{fusc}(j)+\mathrm{fusc}(j+1))$ and $x_{2j+1} = x_j + 1$.*

Proof. The first identity is $x_{2j} = \mathrm{fusc}(2j)/\mathrm{fusc}(2j+1)$ rewritten with the recursion. For the second, $\mathrm{fusc}(2j+2) = \mathrm{fusc}(j+1)$, which is positive by Lemma 3, so $x_{2j+1} = (\mathrm{fusc}(j)+\mathrm{fusc}(j+1))/\mathrm{fusc}(j+1) = x_j + 1$. □

So for $x = x_j$ we have $x_{2j} = x/(1+x)$ and $x_{2j+1} = 1+x$, the two children of x in the Calkin–Wilf tree. Fix m and put $N = 2^m$. For $0 \le i < N$ let

$$D_i = \mathrm{fusc}(N+i) + \mathrm{fusc}(N+i+1), \qquad g_i = \mathrm{fusc}(N+i)/D_i, \qquad f_i = \mathrm{fusc}(N+i+1)/D_i.$$

By Lemma 3, $D_i \ge \mathrm{fusc}(N+i+1) > 0$, and $g_i = x_{2(N+i)}$ by Lemma 6.

Lemma 7 (`row_left_sum`)**.** $\sum_{i=0}^{N-1} g_i = N/2 = 2^m/2$.

Proof. As $D_i > 0$, $g_i + f_i = 1$ for each i, so $\sum_{i<N} g_i + \sum_{i<N} f_i = N$. It remains to show $\sum_{i<N} g_i = \sum_{i<N} f_i$. Reversing the order of summation, $\sum_{i<N} g_i = \sum_{i<N} g_{N-1-i}$. For $0 \le i < N$ put $i^* = N-1-i$. Since $i^* + (i+1) = N$ and $(i^*+1)+i = N$, Lemma 4 gives $\mathrm{fusc}(N+i^*) = \mathrm{fusc}(N+i+1)$ and $\mathrm{fusc}(N+i^*+1) = \mathrm{fusc}(N+i)$. Hence $D_{i^*} = D_i$ and $g_{i^*} = f_i$, so $\sum_{i<N} g_i = \sum_{i<N} f_i = N/2$. □

The same two identities give $x_{N+i^*} = 1/x_{N+i}$: read backwards, a level of the Calkin–Wilf tree is the level of reciprocals, and Lemma 7 says $\sum x/(1+x) = \sum 1/(1+x)$ over a level, by pairing x with $1/x$.

Lemma 8 (`row_sum`)**.** *For every $m \in \mathbb{N}$, the row sum $T(m) = \sum_{i=0}^{2^m-1} x_{2^m+i}$ equals $(3\cdot 2^m-1)/2$.*

Proof. Induction on m. For $m = 0$, $T(0) = x_1 = \mathrm{fusc}(1)/\mathrm{fusc}(2) = 1$. For the step, write $2^{m+1} = 2\cdot 2^m$ and split the sum into even and odd indices (the lemma `sum_range_double`):

$$T(m+1) = \sum_{i=0}^{2^m-1} \left(x_{2(2^m+i)} + x_{2(2^m+i)+1}\right) = \sum_{i=0}^{2^m-1} \left(g_i + x_{2^m+i} + 1\right)$$

by Lemma 6, with $N = 2^m$. By Lemma 7 and the induction hypothesis, $T(m+1) = \frac{2^m}{2} + \frac{3\cdot 2^m-1}{2} + 2^m = \frac{3\cdot 2^{m+1}-1}{2}$. □

Lemma 9 (`prefix_sum`)**.** *For every $M \in \mathbb{N}$, $S(M) = \sum_{k=0}^{2^M-1} x_k = (3\cdot 2^M - 3 - M)/2$.*

Proof. Induction on M. For $M = 0$, $S(0) = x_0 = \mathrm{fusc}(0)/\mathrm{fusc}(1) = 0$. As $2^{M+1} = 2^M + 2^M$, the indices below 2^{M+1} are those below 2^M followed by $2^M + i$, $0 \le i < 2^M$, so by Lemma 8, $S(M+1) = S(M) + T(M) = \frac{3\cdot 2^M - 3 - M}{2} + \frac{3\cdot 2^M - 1}{2} = \frac{3\cdot 2^{M+1} - 3 - (M+1)}{2}$. □

Proposition 10. *For every $k \ge 0$, $s(2\cdot 4^k - 1) = 3\cdot 4^k - k - 2$ and $s(2\cdot 4^k - 2) = 3\cdot 4^k - 3k - 3$.*

Proof. Let $k \ge 0$ and $M = 2k+1$, so that $2\cdot 4^k = 2^M \ge 2$. By Lemma 9,

$$S(M) = \frac{3\cdot 2^{2k+1} - 3 - (2k+1)}{2} = \frac{6\cdot 4^k - 2k - 4}{2} = 3\cdot 4^k - k - 2.$$

If $n = 2\cdot 4^k - 1$, then $n+1 = 2^M$ and $s(n) = S(M) = 3\cdot 4^k - k - 2$. If $n = 2\cdot 4^k - 2$, then $n+1 = 2^M - 1$; splitting off the last term of $S(M)$ and using Lemma 5,

$$S(M) = s(n) + x_{2^M-1} = s(n) + \mathrm{fusc}(2^M - 1)/\mathrm{fusc}(2^M) = s(n) + M,$$

so $s(n) = 3\cdot 4^k - k - 2 - (2k+1) = 3\cdot 4^k - 3k - 3$. □

Proof of Theorem 2. Let $k \ge 0$ and let $n = 2\cdot 4^k - 1$ or $n = 2\cdot 4^k - 2$. By Proposition 10, $s(n)$ is (the image in $\mathbb{Q}$ of) an integer, whose reduced denominator is 1 (Mathlib's `Rat.den_intCast`). Hence $a(n) = 1$. □

In Lean, the two values of Proposition 10 appear as intermediate equalities, cast from $\mathbb{Z}$ to $\mathbb{Q}$, inside the proof of `result`; the exported theorem is only the statement $a(n) = 1$ of Theorem 2.

Remark 11. The following is not part of the conjecture and is not formalized. For $M \ge 1$ the numerator $3\cdot 2^M - 3 - M$ in Lemma 9 is even exactly when M is odd. Hence for even $M \ge 2$ both $s(2^M - 1) = S(M)$ and $s(2^M - 2) = S(M) - M$ are halves of odd integers, so $a(2^M - 1) = a(2^M - 2) = 2$, as in $a(2) = a(3) = a(14) = a(15) = 2$. One of the math0 referees computed $a(n)$ exactly for all $n < 131077$ and found $a(n) = 1$ only for n of the form $2\cdot 4^k - 1$ or $2\cdot 4^k - 2$; we do not prove the converse of the conjecture in general.

4. Machine verification

Files and declarations. The proof is in `lean/Math0/Frontier/OEIS/A355076.lean` in the math0 repository, `https://github.com/Foundation-0/math0`. It imports the frozen statement file (Section 2) and the audit commands in `lean/Math0/Audit.lean`. Its private lemmas are `fusc_pos`, `fusc_symm`, `fusc_two_pow`, `fusc_two_pow_sub_one`, `sum_range_double`, `ratio_even`, `ratio_odd`, `row_left_sum`, `row_sum` and `prefix_sum` (Lemmas 3–9; Proposition 10 and Theorem 2 are proved inside `result`). The main ones read (signatures verbatim, proofs omitted):

```
namespace Math0.Frontier.A355076
open Math0.Statements.A355076
private lemma fusc_symm : ∀ m i j, i + j = 2 ^ m → fusc (2 ^ m + i) = fusc (2 ^ m + j)
private lemma row_left_sum (m : ℕ) :
    ∑ i ∈ Finset.range (2 ^ m),
      (fusc (2 ^ m + i) : ℚ) / (fusc (2 ^ m + i) + fusc (2 ^ m + i + 1)) = 2 ^ m / 2
private lemma row_sum (m : ℕ) :
    ∑ i ∈ Finset.range (2 ^ m), (fusc (2 ^ m + i) : ℚ) / fusc (2 ^ m + i + 1) =
      (3 * 2 ^ m - 1) / 2
private lemma prefix_sum (M : ℕ) :
    ∑ k ∈ Finset.range (2 ^ M), (fusc k : ℚ) / fusc (k + 1) = (3 * 2 ^ M - 3 - M) / 2
theorem result : statement
end Math0.Frontier.A355076
#math0_settles Math0.Frontier.A355076.result Math0.Statements.A355076.statement
```

The main declaration is `Math0.Frontier.A355076.result`, of type `Math0.Statements.A355076.statement`; in `row_sum` and `prefix_sum` the subtractions are in $\mathbb{Q}$. Lemma 7 uses Mathlib's `Finset.sum_range_reflect` to reverse the order of summation, Lemma 9 uses `Finset.sum_range_add`, and the last step uses `Finset.sum_range_succ` and `Rat.den_intCast`; the arithmetic is done by `omega`, `ring`, `norm_num`, `field_simp`, `push_cast` and `linarith`.

The proof involves no large computation. Its only kernel evaluations are `decide +kernel` proofs of the base cases $\mathrm{fusc}(0) = 0$, $\mathrm{fusc}(1) = 1$ and $\mathrm{fusc}(2) = 1$ (directly in Lemmas 4, 5, 8, and through `fusc_zero` and `fusc_one`); for each, the kernel reduces the decision procedure for the equation to `true`, unfolding `fuscAux` a few times. The data checks `fusc_data` and `a_data` are evaluated by the kernel in the same way (the statement file compiles in about 13 seconds), but the proof does not use them. Neither file uses `native_decide`, `sorry` or a `maxHeartbeats` option.

Audit. The file ends with `#math0_settles`, defined in `lean/Math0/Audit.lean` for statements that math0 formalizes itself. It fails unless (1) `statement` is a definition of type `Prop` without universe parameters; (2) `statement` and `result` depend on no axioms other than `propext`, `Classical.choice` and `Quot.sound` (this rejects `sorryAx`, `Lean.ofReduceBool`, `Lean.trustCompiler` and custom axioms); and (3) the type of `result` is syntactically the statement or its negation, after removing metadata annotations, with no unfolding. On this file it reports

```
math0 audit: 'Math0.Statements.A355076.statement' depends on axioms [Classical.choice, Quot.
    sound, propext]
math0 audit: 'Math0.Frontier.A355076.result' depends on axioms [Classical.choice, Quot.sound,
    propext]
math0 statement check: 'Math0.Frontier.A355076.result' proves 'Math0.Statements.A355076.
    statement'
```

Toolchain and reproduction. We use Lean 4 [6] `v4.33.1` and Mathlib [5] `v4.33.1`, pinned in `lean/`; the repository also pins formal-conjectures [4] at commit `c77fdd4`, which this result does not use. To reproduce, run `scripts/verify.sh` from the repository root: it fetches the Mathlib build cache, builds all math0 results with `lake build` (re-running every audit command), checks that the pinned `formal_conjectures` and `mathlib` checkouts are unmodified, and reports the number of passed statement checks. The single file can be checked with `lake env lean Math0/Frontier/OEIS/A355076.lean` in the directory `lean`; this took about 16 seconds on our machine.

References

[1] N. Calkin and H. S. Wilf, *Recounting the rationals*, Amer. Math. Monthly **107** (2000), no. 4, 360–363.

[2] K. S. Choudary and A. S. Reddy, *Calkin–Wilf tree*, arXiv:2003.13466 [math.CO], 2020 (accepted for publication in Resonance, according to the arXiv record).

[3] E. W. Dijkstra, *Selected Writings on Computing: A Personal Perspective*, Springer, 1982, p. 232.

[4] M. Firsching, P. Lezeau, S. Mercuri, M. Z. Horváth, Y. Dillies, C. Sönne, E. Wieser, F. Zhang, T. Hubert, B. Agüera y Arcas, and P. Kohli, *Formal Conjectures: An Open and Evolving Benchmark for Verified Discovery in Mathematics*, arXiv:2605.13171 [cs.AI], 2026. Repository: `https://github.com/google-deepmind/formal-conjectures`.

[5] The mathlib Community, *The Lean mathematical library*, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, 367–381.

[6] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction – CADE 28, Lecture Notes in Comput. Sci. **12699**, Springer, 2021, 625–635.

[7] S. Northshield, *Stern's diatomic sequence* $0, 1, 1, 2, 1, 3, 2, 3, 1, 4, \ldots$, Amer. Math. Monthly **117** (2010), no. 7, 581–598.

[8] M. A. Stern, *Über eine zahlentheoretische Funktion*, J. Reine Angew. Math. **55** (1858), 193–220.

[9] The OEIS Foundation Inc., `https://oeis.org/A002487` (Stern's diatomic series), comment by A. Adamchuk (2006).
[10] The OEIS Foundation Inc., `https://oeis.org/A052940`, comment by C. Najafi (2011).
[11] The OEIS Foundation Inc., `https://oeis.org/A355075`, entry by R. Sigrist (2022).
[12] The OEIS Foundation Inc., `https://oeis.org/A355076`, entry by R. Sigrist (2022).

Produced by Claude (an AI model developed by Anthropic) in the math0 project